

Множества
set

Применяем, где необходимо, чтобы все элементы, которые храним, были уникальными и не требовался порядок.

СЦ **Цехош София Ивановна**

Дата обращения 16.10.2024

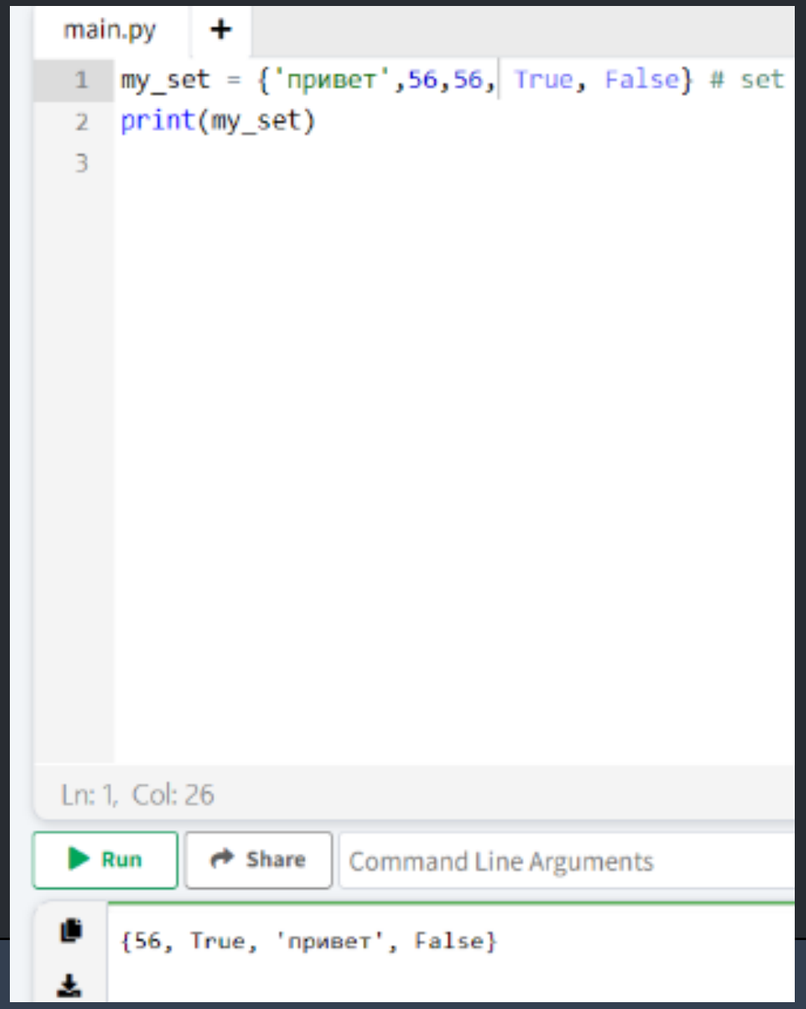
set

Множество – это неупорядоченный набор уникальных элементов. **Множество не может** содержать **два одинаковых элемента**, все элементы множества уникальны.

```
my_set = {'привет',56,56, True, False} # set
print(my_set)

#Вывод
{'привет', True, 56, False}
```

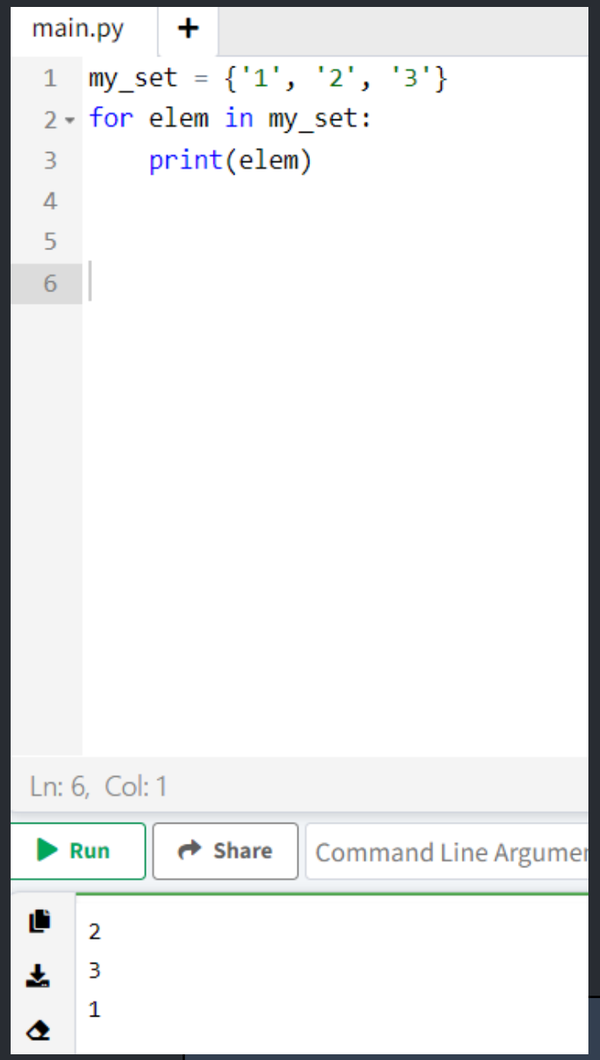


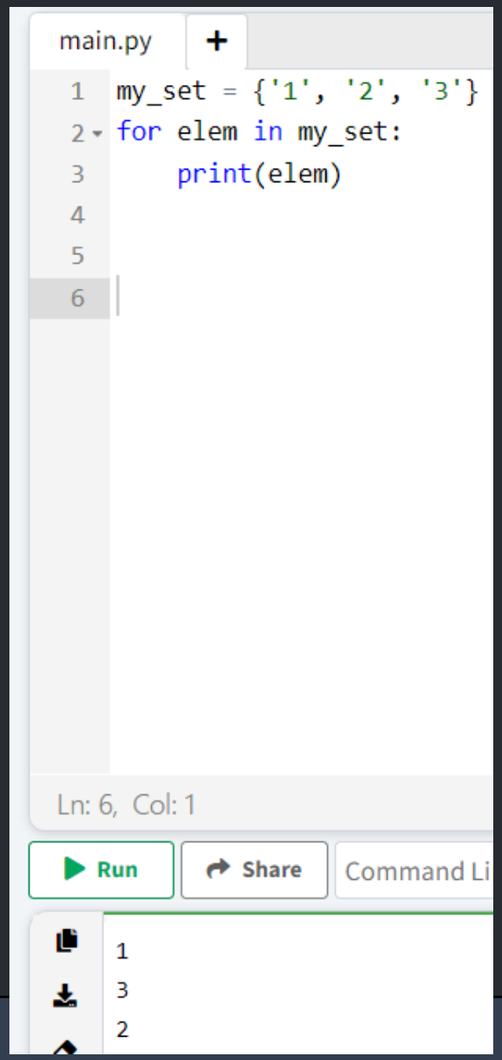


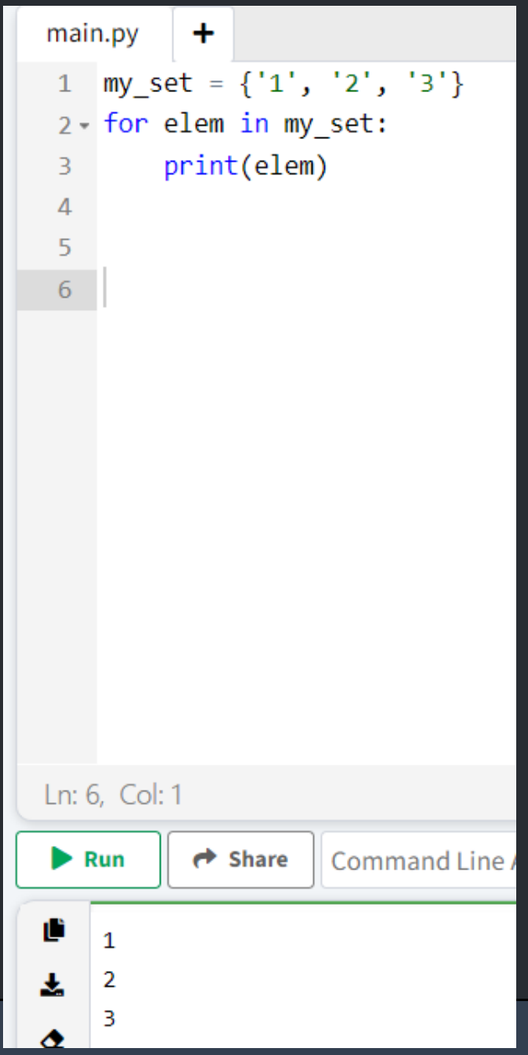
Функция set()

Мы можем перебрать все элементы множества, но поскольку множество является **неупорядоченной** структурой, **порядок вывода** этих элементов **всегда будет разным**.

```
my_set = {'1', '2', '3'}
for elem in my_set:
    print(elem)
```







Функция set()

Пустое множество my_set = **set()**

Применение пустого множества:

1. **Инициализация переменной**: Это просто инициализирует переменную my_set как пустое множество. В некоторых случаях это может быть полезно для предотвращения ошибок во время выполнения программы, когда переменная используется до того, как ей присваивается значение.

2. Проверка условий: Если вам нужно проверить, является ли множество пустым, можно сравнить его с пустым множеством:

```
if my_set == set():
    print("Множество пусто.")
else:
    print("Множество содержит элементы.")
```

3. Удаление всех элементов из множества: Пустое множество можно использовать для удаления всех элементов из существующего множества:

```
my_set = {1, 2, 3}
my_set.clear() # Удаляет все элементы из множества
assert(my_set == set())
```

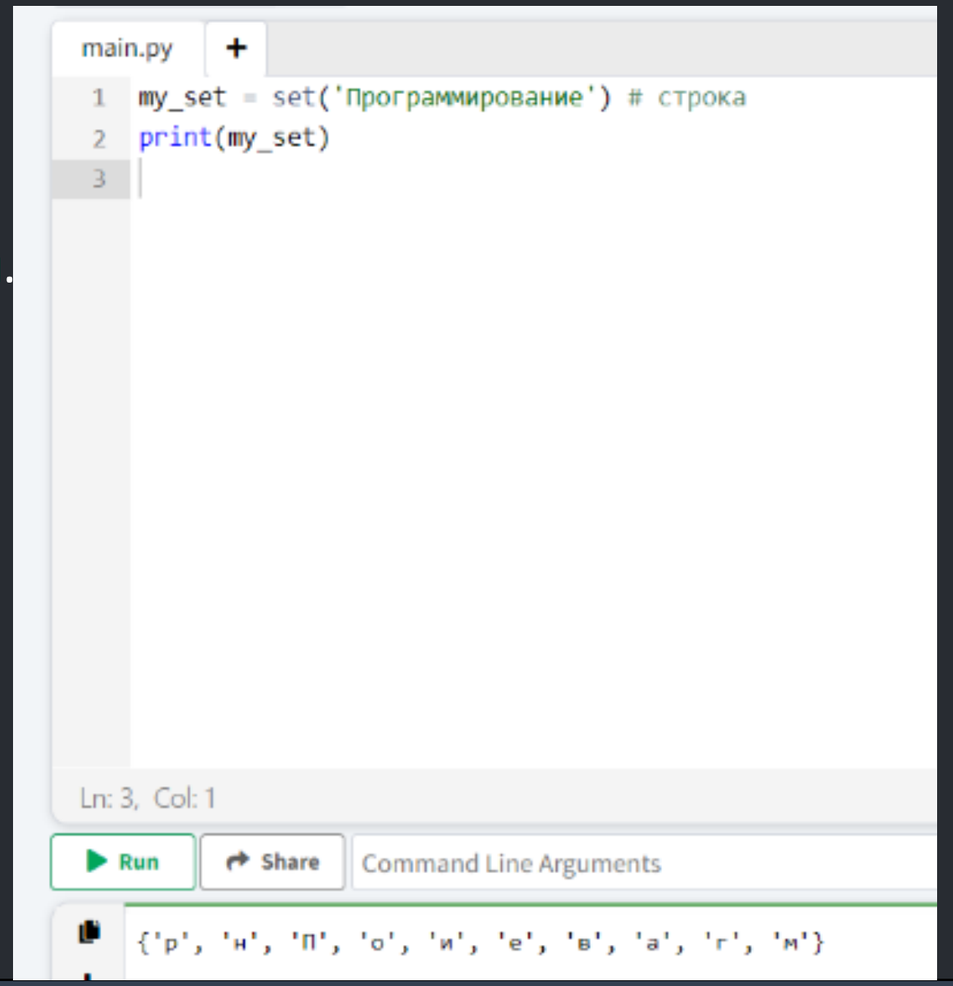
Функция set()

Функция set() может преобразовывать во множество другие типы данных:

- ✓ Строки
- ✓ Кортежи
- ✓ Списки

Внимание! Во множестве останутся **только уникальные элементы**.

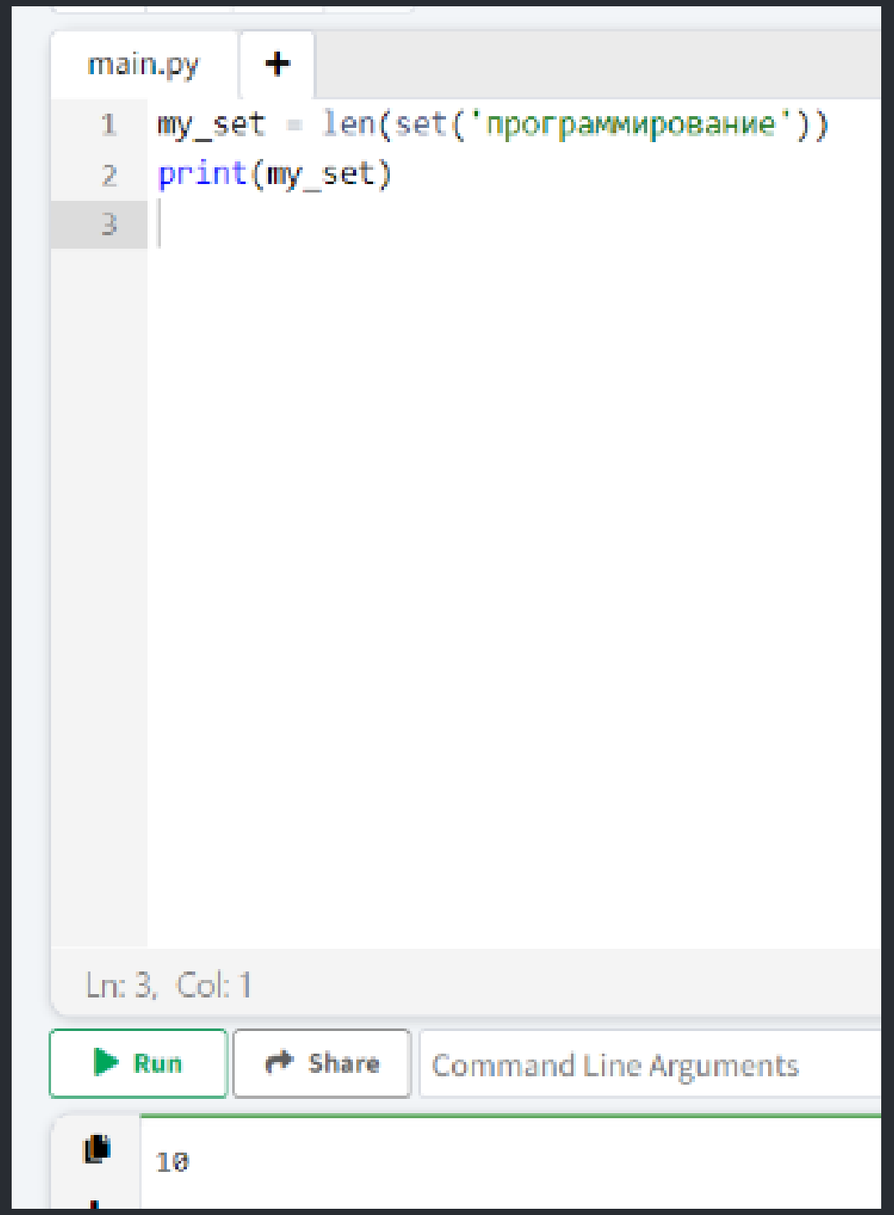
```
my_set = set('программирование') # строка
print(my_set)
# Вывод
{'р', 'н', 'п', 'о', 'и', 'е', 'в', 'а', 'г', 'м'}
```



Пример

```
my_set = len(set('программирование'))
print(my_set)


# Вывод
10
```



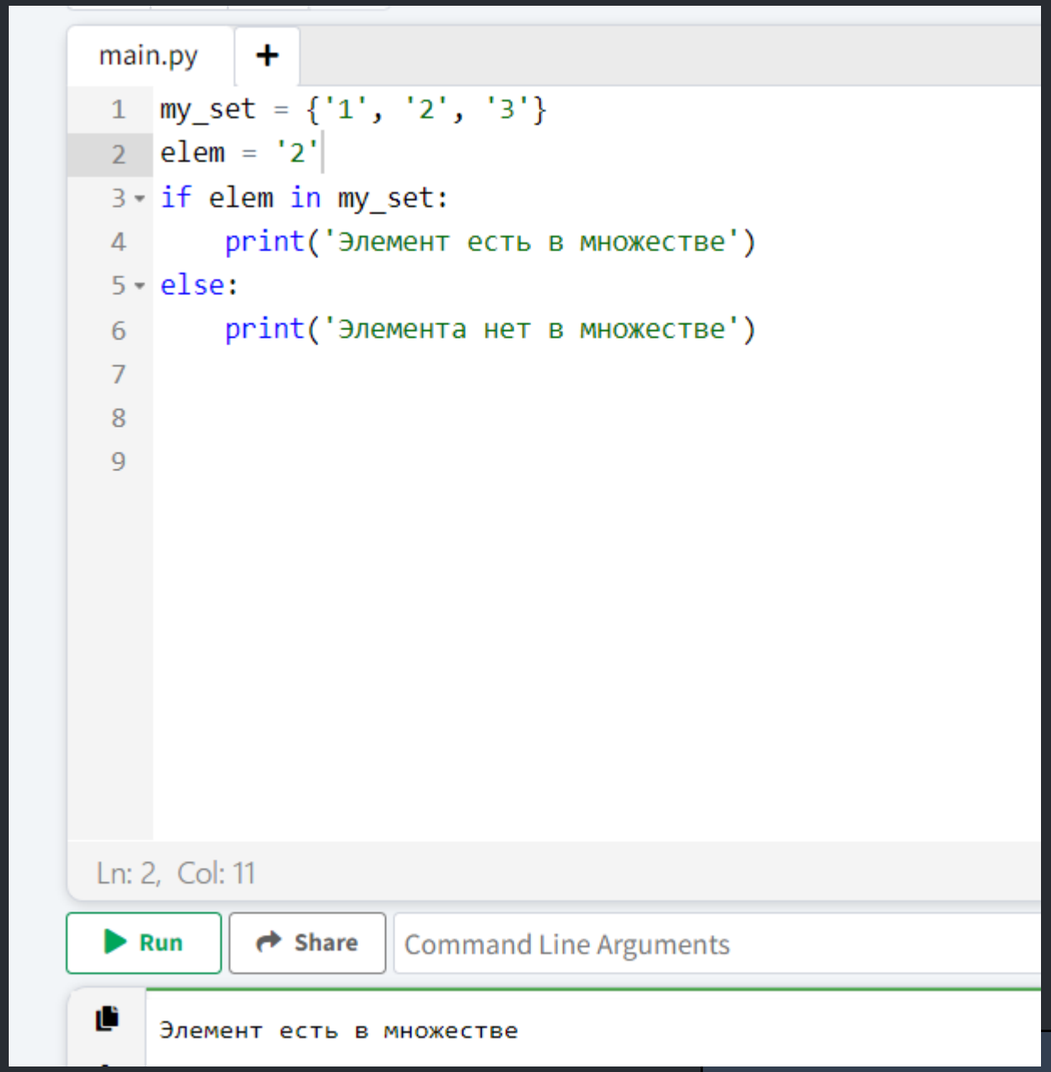
Функция set()

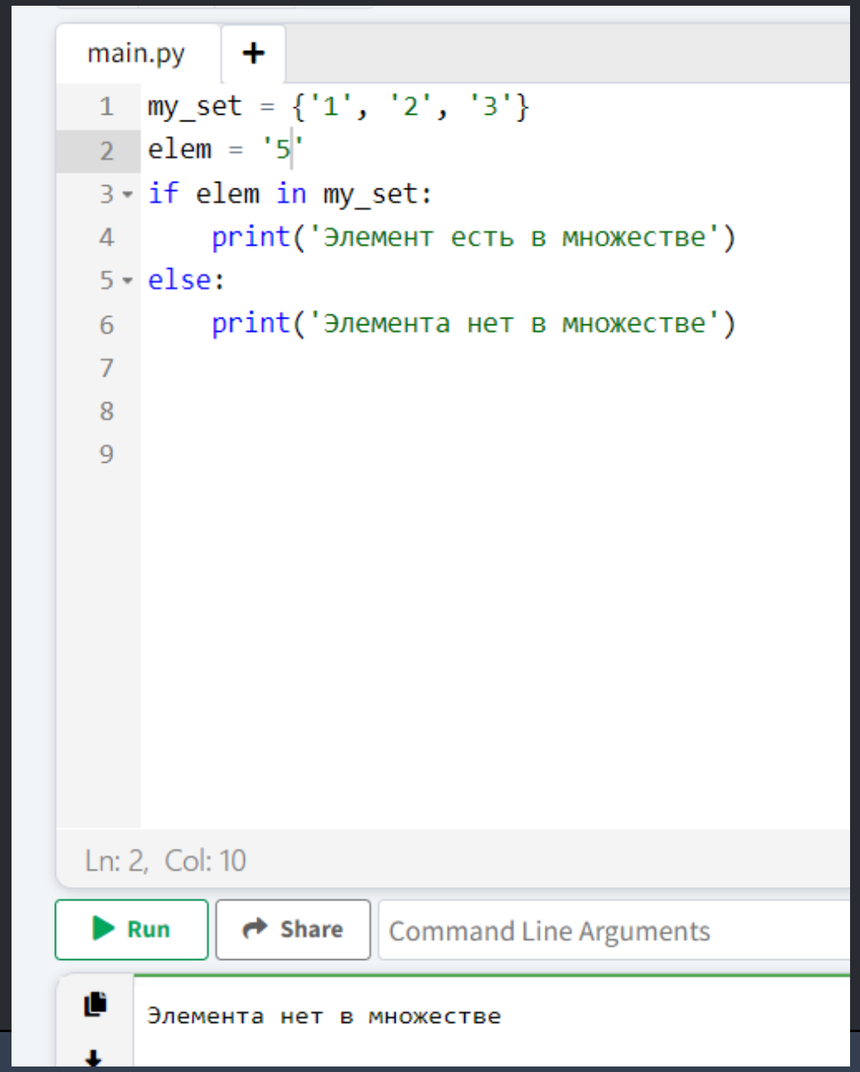
```
my_list = ['привет',56, 56, True, False]# список
my_set = set(my_list) # set используется для преобразования другой  структуры данных (списка или кортежа) в  множество
print(my_set)
#Вывод
{56, True, 'привет', False}
```



Функция set()

Проверить наличие элемента в множестве



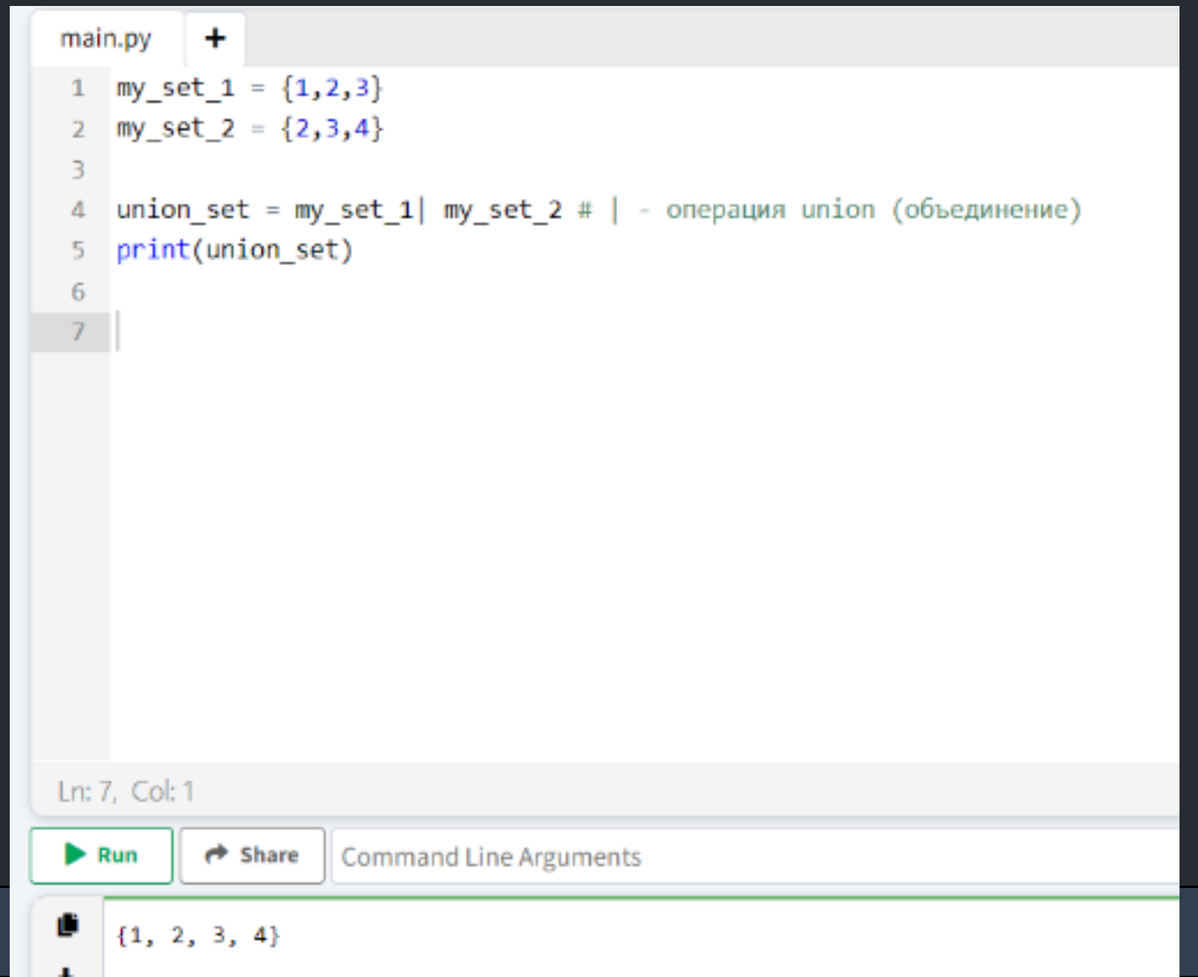


Операции над множеством

```
my_set_1 = {1,2,3}
my_set_2 = {2,3,4}


union_set = my_set_1| my_set_2 # оператор | (union объединение)
print(union_set)


#Вывод
{1, 2, 3, 4} # остаются только уникальные элементы
```



my_set_1 = {1,2,3}

my_set_2 = {2,3,4}

Операции над множеством

intersection_set = my_set_1 **&** my_set_2 # оператор **&** (**intersection** - пересечение)

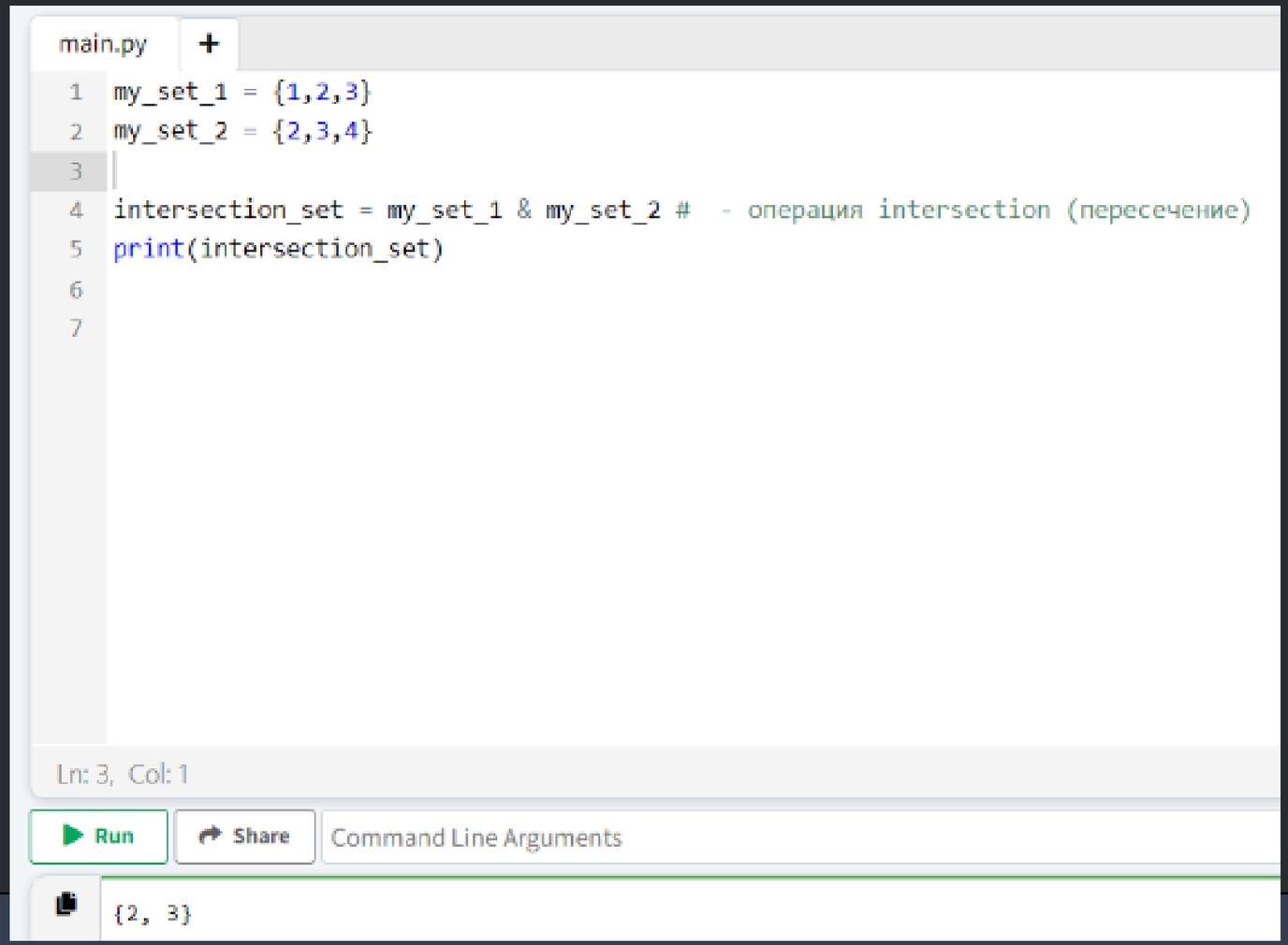
print(intersection_set)

#Вывод

{2, 3}

Результат:

& - это элементы, которые есть в обоих множествах.



Операции над множеством

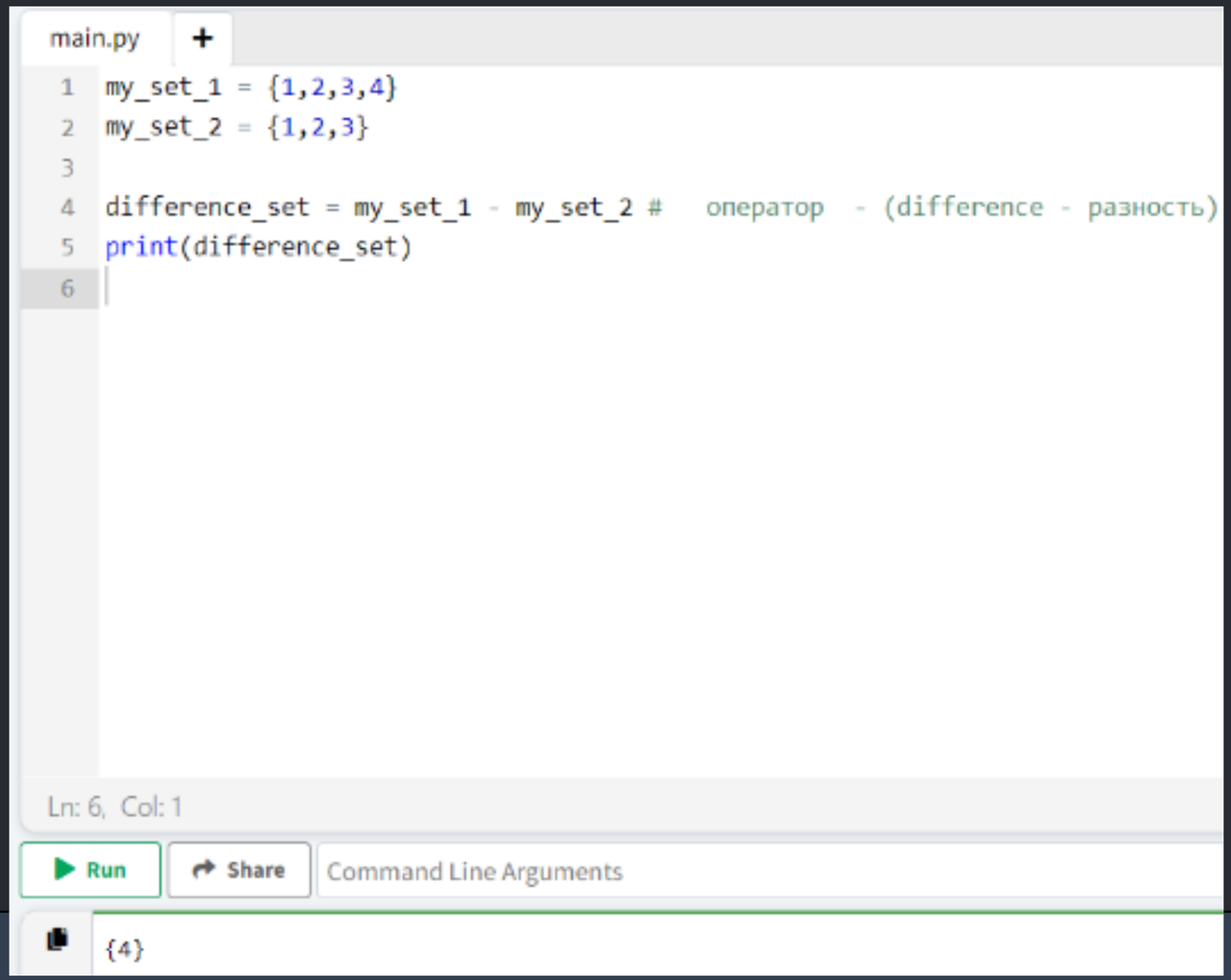
```
my_set_1 = {1,2,3,4}
my_set_2 = {1,2,3}

difference_set = my_set_1 - my_set_2 #   оператор  - (difference - разность)
print(difference_set)

#Вывод
{4}
```

Результат:

оператор **–** удалит из множества my_set_1 элементы, которые существуют и во множестве my_set_1, и во множестве my_set_2.



Операции над множеством

my_set_1 = {1,2,3}

my_set_2 = {2,3,4}

symmetric_difference_set = my_set_1 ^ my_set_2 # оператор ^ (**symmetric_difference** - симметрическая разность)

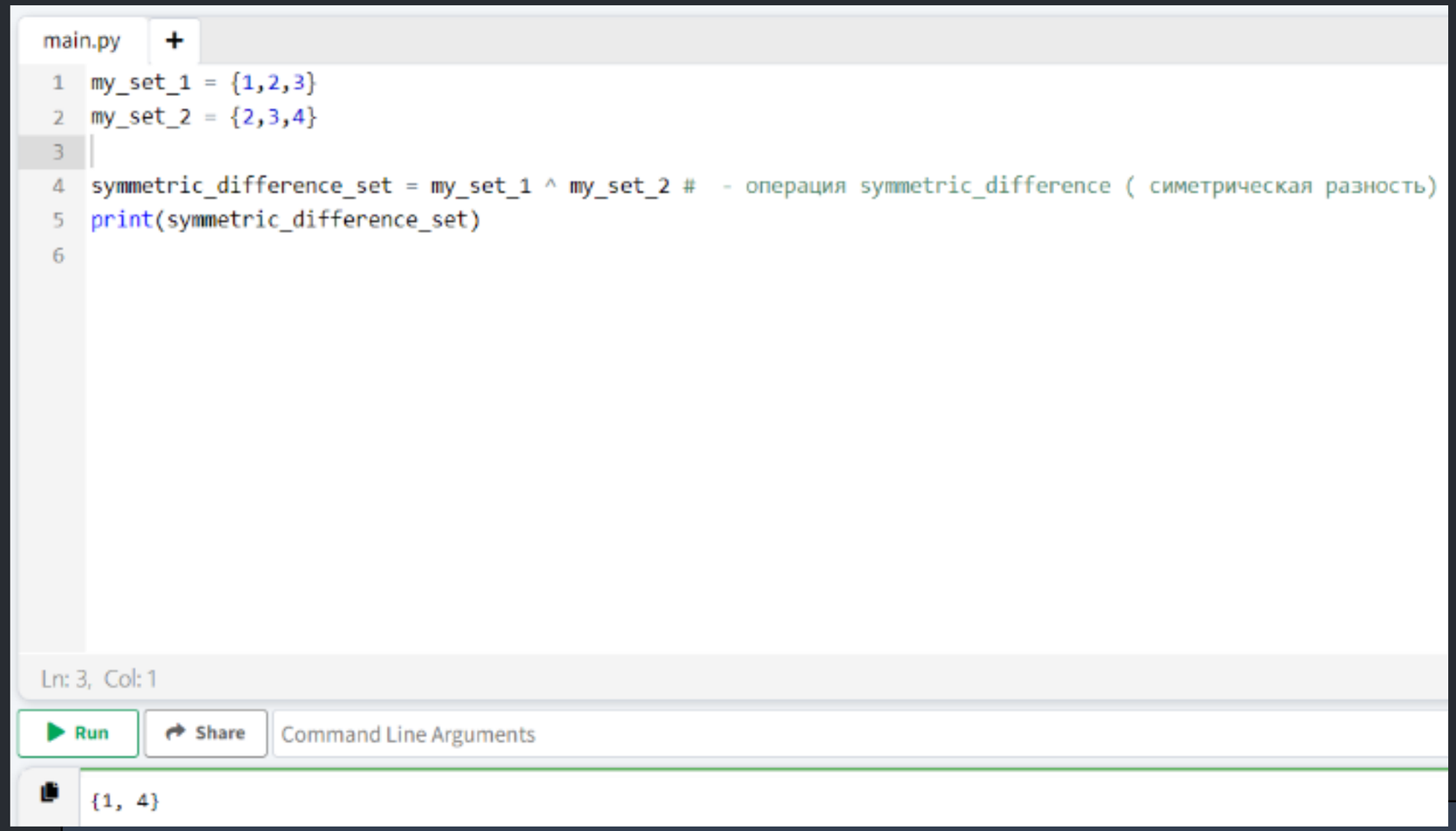
print(symmetric_difference_set)

#Вывод

{1, 4}

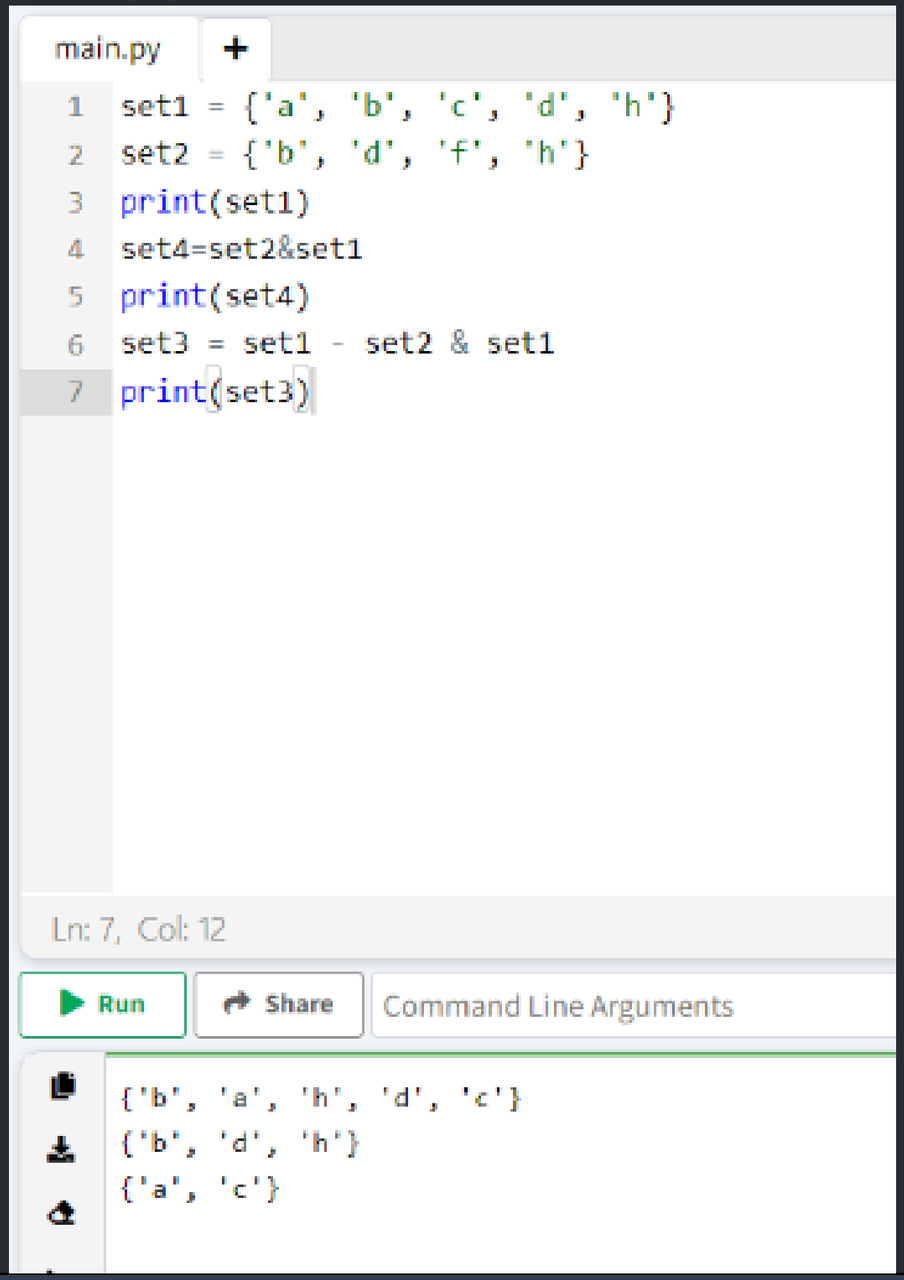
Результат:

оператор ^ возвращает элементы обоих множеств, исключая одинаковые элементы.



Операции над множеством

```
set1 = {'a', 'b', 'c', 'd', 'h'}
set2 = {'b', 'd', 'f', 'h'}
print(set1)
set4=set2&set1
print(set4)
set3 = set1 - set2 & set1
print(set3)
```



Методы множеств:

add(<Элемент>) – добавляет <Элемент> во множество

```
1 literature_set = {'Мастер и Маргарита', 'Герой нашего времени','Ревизор'}
2 elem = input('Введите название произведения, которое хотите добавить')
3 literature_set.add(elem)
4 print('Множество:')
5 print(literature_set)
```

Ln: 1, Col: 64

Stop | Share | Command Line Arguments

```
Введите название произведения, которое хотите добавить
```

```
Введите название произведения, которое хотите добавить
Двенадцать стульев
```

```
1 literature_set = {'Мастер и Маргарита', 'Герой нашего времени','Ревизор'}
2 elem = input('Введите название произведения, которое хотите добавить')
3 literature_set.add(elem)
4 print('Множество:')
5 print(literature_set)
```

Ln: 1, Col: 64

Run | Share | Command Line Arguments

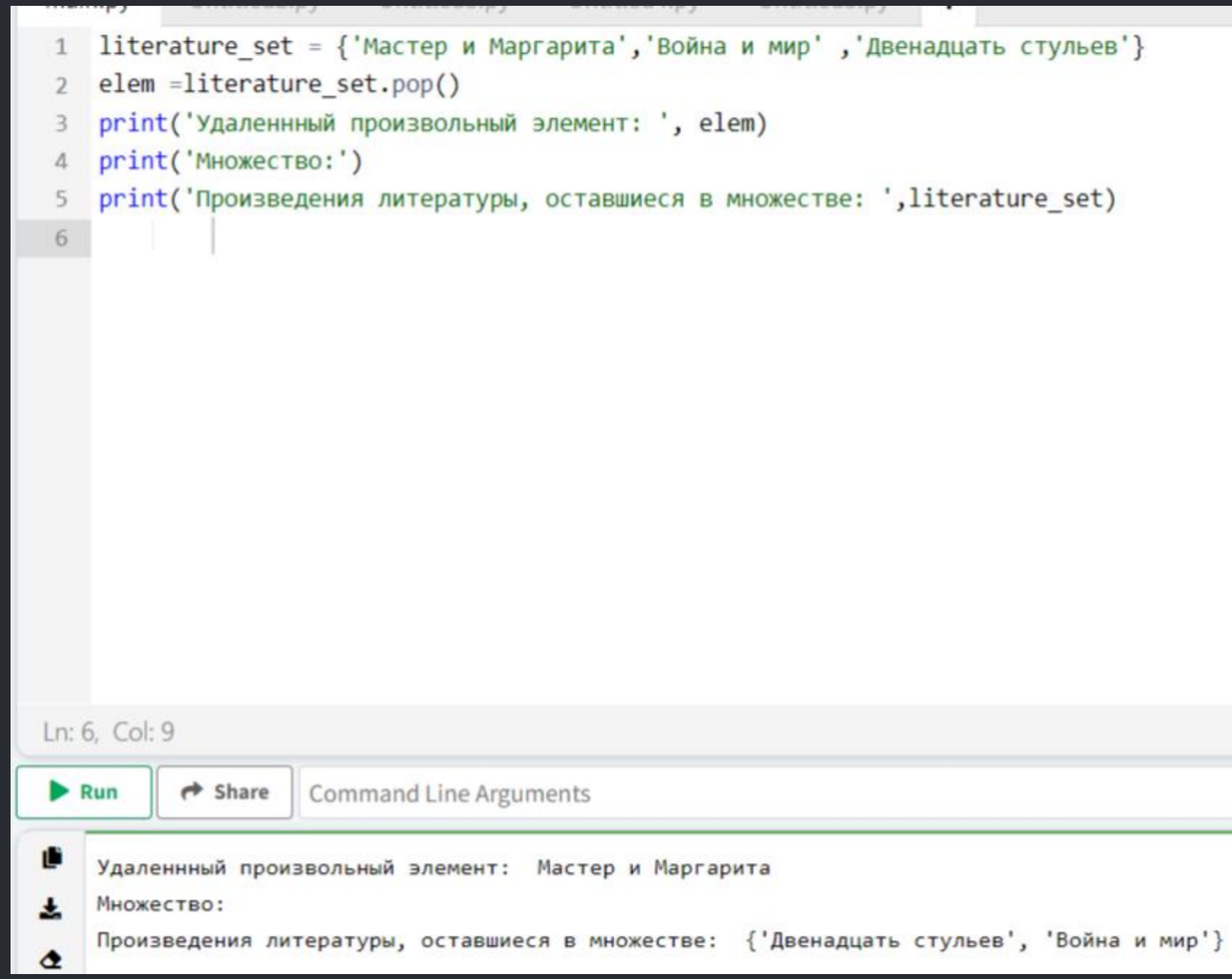
```
Введите название произведения, которое хотите добавить
Двенадцать стульев
Множество:
{'Двенадцать стульев', 'Ревизор', 'Мастер и Маргарита', 'Герой нашего времени'}
```

literature_set = {'Мастер и Маргарита', 'Герой нашего времени','Ревизор'}
elem = input('Введите название произведения, которое хотите добавить')
literature_set.add(elem)
print('Множество:')
print(literature_set)

Методы множеств:

pop() – удаляет произвольный элемент и возвращает его.

```
literature_set = {'Мастер и Маргарита','Война и мир' ,'Двенадцать стульев'}
elem =literature_set.pop()
print('Удаленнный произвольный элемент: ', elem)
print('Множество:')
print('Произведения литературы, оставшиеся в множестве: ',literature_set)
```



Методы множеств:

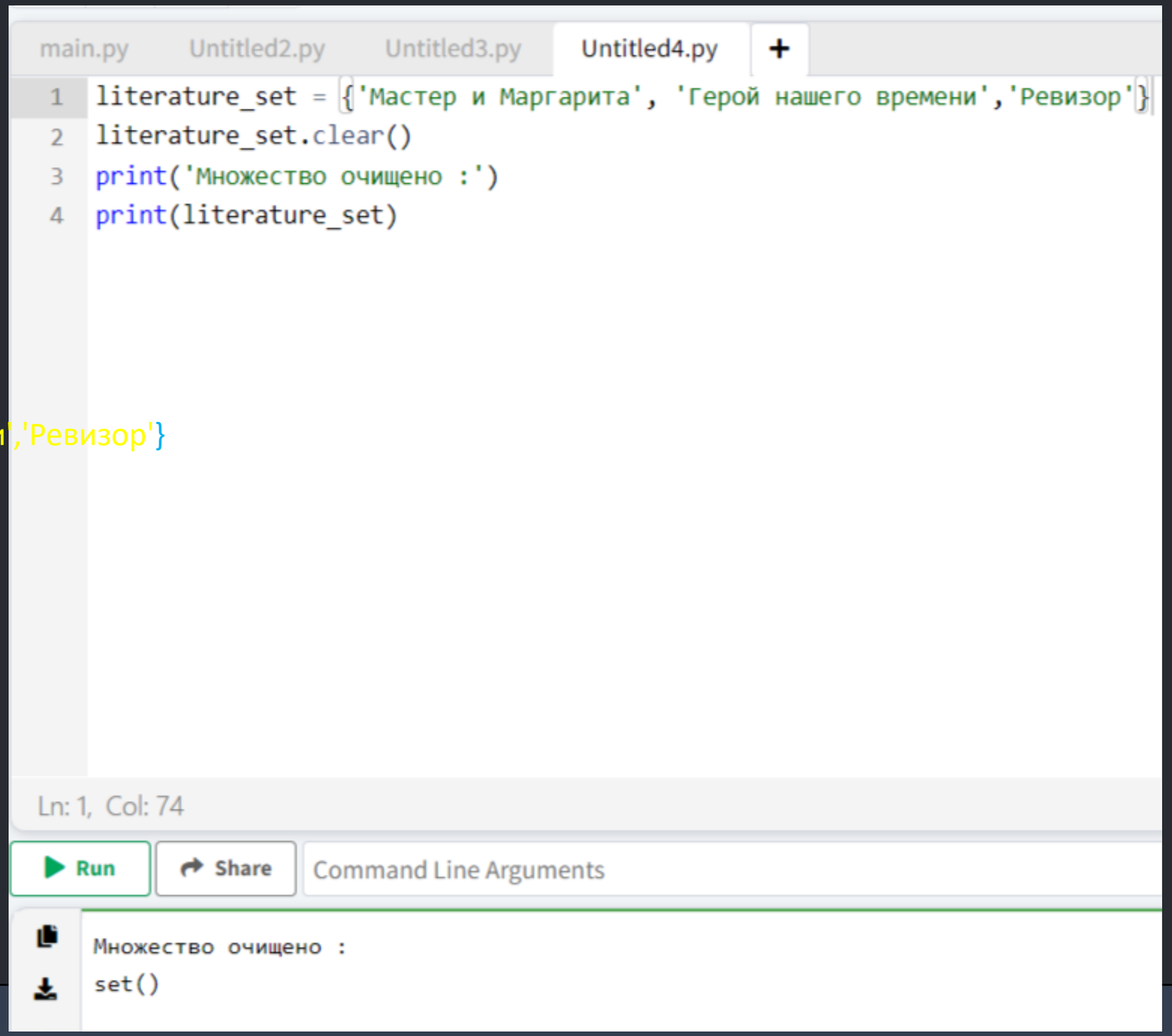
clear() – очищает множество

literature_set = {'Мастер и Маргарита', 'Герой нашего времени','Ревизор'}

literature_set.clear()

print('Множество очищено :’)

print(literature_set)



Методы множеств:

remove(<– удаляет <Элемент> из множества

discard(<Элемент>) – удаляет указанный элемент из множества

Методы remove() и discard() отличаются тем, что если указанный элемент отсутствует во множестве, то в первом случае будет возвращена ошибка, а во втором никаких сообщений не будет.

Методы множеств:

remove(<– удаляет <Элемент> из множества

